Document Insight Engine

Question 1 — PDF Q&A; with Highlighted Sources

A full-stack PDF question-answering application that lets users upload a PDF, ask questions, receive grounded answers, and inspect supporting passages highlighted directly on the original PDF.

1. Implementation Approach

PDF processing: PyMuPDF extracts pages, readable text, and word-level coordinates. The system validates file type, size, and readable text content.

Chunking: Extracted words are grouped into overlapping chunks of 180 words with a 60-word overlap. Each chunk retains its page, text, words, coordinates, and embedding.

Retrieval: A lightweight deterministic hashing-based embedding represents tokens and bigrams. Retrieval combines semantic similarity, keyword matching, and a requirement boost using 0.65 semantic + 0.25 keyword + 0.10 boost.

Answer generation: Retrieved passages are supplied to the Groq language model. It is instructed to answer only from the supplied sources, cite support, and report when the document lacks enough information.

Highlighting: Supporting text is mapped back to original words. Their PDF coordinates are converted into rectangles and drawn over the PDF in the browser. Citations navigate to the corresponding highlight.

2. Evaluation Results

The deployed application was evaluated using 10 questions across two PDF documents. Each question was checked separately for retrieval correctness and highlight correctness.

Metric	Result
Questions tested	10
Correct supporting passage retrieved	10/10 (100%)
Correct highlight location	10/10 (100%)

3. Text-to-Position Mapping

The PDF processor preserves word-level coordinates during extraction. Each word contains text and a bounding box (x0, y0, x1, y1). Supporting passages are mapped back to the original words, and each bounding box is converted to a browser highlight rectangle. This places the highlight directly over the original PDF text rather than using only a page number or a downloaded annotation.

Pipeline: PDF → word extraction + coordinates → overlapping chunks → retrieval → supporting words → coordinate rectangles → browser highlight → citation navigation.

4. Difficult Layouts and Limitations

• Scanned/image-only PDFs without a readable text layer are rejected.
• Complex multi-column layouts can cause visual and extracted reading order to differ.
• Tables and heavily formatted documents may be harder to map reliably.
• Rotated or transformed text can be difficult to position.
• Text split across unusual regions may affect highlighting.

5. Challenges and Solutions

Deployment memory: The initial transformer embedding model exceeded the available deployment memory. It was replaced with a lightweight deterministic hashing-based embedding implementation.

Cross-origin deployment: The Vercel frontend and Render backend use different origins. FastAPI CORS middleware was configured for the production frontend origin.

Reliable highlighting: Word-level coordinates were preserved so supporting passages could be highlighted directly on the PDF.

6. Error Handling

The application handles invalid file types, empty files, oversized PDFs, corrupted/unreadable PDFs, and scanned/image-only PDFs without breaking. If a question cannot be answered from the document, the system reports that there is not enough information rather than guessing or highlighting unrelated text.

7. Deployment

Frontend: https://document-insight-engine-frontend.vercel.app/app
Backend: https://document-insight-engine-backend-5.onrender.com
API documentation: https://document-insight-engine-backend-5.onrender.com/docs

8. Future Improvements

• Implement true token-level server streaming using Server-Sent Events.
• Add OCR support for scanned PDFs.
• Use stronger semantic embeddings when more resources are available.
• Improve multi-column, table, rotated, and complex layout handling.
• Use persistent object storage for documents.
• Improve multi-line highlight grouping and passage ranking.

9. Requirement Coverage

Requirement	Status
50+ page documents	Completed
PDF displayed in browser	Completed
Direct text highlighting	Completed
Supporting passages highlighted	Completed
Unsupported questions handled without guessing	Completed
Oversized/scanned/invalid PDF handling	Completed
10-question evaluation	Completed
Retrieval accuracy measured	Completed
Highlight accuracy measured	Completed
Text-to-position approach documented	Completed
Layout limitations documented	Completed

Streaming note: The current production backend returns the completed answer as JSON; the frontend exposes a compatibility streaming interface. True token-by-token server streaming is identified as a future improvement.